

MALDA: Game Production Report

CSCI 4805 – Capstone

Dr. J. Nocholson

May 5th, 2026

Team Members

Anthony Chapman

Eric Moore

Rachel Mohamed

Noah Turrock

Introduction

Abstract

Malda is a 2D Puzzle-Platforming game that sees players navigate platforms, fight enemies, and overcome obstacles to reach their goal. Upon completing a level, players will be awarded a new item to help them continue their journey. Some levels require items from previous levels to be completed. Levels may include optional portions that require items from later in the game to discover. Malda will feature a minimalistic pixel art style.

Malda players will need to be creative with their use of items, sometimes combining them, to succeed. As players make it further in the game, they will find their technical platforming skills tested.

Purpose

Malda seeks to bridge the gap between platforming games and puzzle games. The fusion of these genres is familiar and approachable for players of all skill levels. Players are rewarded for exploring new areas with new items to help them overcome obstacles while facing minimal punishments for their mistakes. Players are encouraged to have fun as they tackle obstacles.

The 2D pixel art style of Malda attracts gamers with its cute, friendly appearance. The art style is modern yet nostalgic for gamers. Through worldbuilding, story hints through gameplay, and its art style, Malda captures gamers' attention and draws them into exploring its world.

Problem Solved

Platforming games today are typically very technically challenging to play and often require frame-perfect inputs to solve harder sections. Casual gamers don't have many options to enjoy platforming games in the same way other gamers do. Puzzle-adventure games offer many options that require basic problem-solving rather than challenging gameplay. Malda brings the approachable puzzle-solving nature of those adventure games to the platforming genre. Puzzle game enthusiasts will be able to enjoy a whole new genre of game without needing to master daunting mechanics.

Technology

Technologies Used

Our team's main platform for the project was built collaboratively in Unity, a game engine. We chose Unity because it handles much of the background work most video game projects require, and we had a short time frame to build our video game. Unity scripts for video game design are written in the programming language C#. This was a non-negotiable for our platform choice.

For the team's version control and to support the required technologies we use, we use GitHub. GitHub is a well-known platform for sharing projects among team members. Each member can make changes either through the main path of the project or through a branch that holds a copy of the project at the time the branch was created. Our team created branches from time to time when we did not want to disrupt or damage the main project, for testing and/or implementing new code.

For communication, our team chose to communicate through our own Discord server primarily. Discord began as a platform for gamers to build communities and has since evolved into a major social platform. This helped us to communicate any changes, bugs, fixes, and game development as time progressed.

In the early stages of development, we attempted to use technologies such as Google Gemini for art generation and Gimp to create sprite sheets or animated objects. We quickly realized that generating art would not be reliable and opted to use resources from CraftPix.net. This helped both save time on art design and allowed us to progress faster with level design and animation.

For custom sprite sheet creation, Procreate was used. Procreate is a mobile app available for tablets and offers a great tool for digital art and layering. Procreate gives us the ability to create an infinite range of sizes, brushes, colors, and layering. This helped to create the player's animations and interactive objects in our game.

Our Background music was commissioned from a third-party member as custom work. The music was made using FL Studio, a mobile app for music editing and production. Each

obstacle's background music for each level was intentionally created to loop and blend well when played as one full track.

Technological Difficulties

Unity was a game engine that very few of us had any experience with. We looked at some tutorials for an idea of how Unity works. Once we had an idea of how Unity works, our coding experience in class helped us speed up development. If we ran into issues understanding a problem or method while coding in Unity, we could easily watch reference videos to learn how to fix it.

GitHub, like Unity, was a program that very few of us had serious experience in. However, GitHub was very easy to learn, and GitHub Desktop made applying updates to the GitHub Repository simple and easy to understand. We did run into some issues when using GitHub and Unity together. Still, GitHub Desktop was so easy to use that we were able to solve most problems in just a few hours, which, given our experience level, was impressive.

Unlike the previous two, Discord was a technology with which pretty much all of us had extensive experience. There was very little learning, if any, in using this technology. This made communicating with everyone much easier and allowed us to complete work on the project outside of class time without risking breaking someone else's work.

Development Difficulties

There were several instances in which our choice of technologies made things equally easy and difficult. As mentioned above, our choice in AI art generation quickly dissolved, as AI could only generate so much to meet our requirements and demands for style and design. For example, we took our original Malda design and asked Gemini to generate a sprite sheet for Malda. When the generation was complete, the image showed several inconsistencies with proportion, movement, and color.

Discord posed no issues at all and made communication easy and readily available. If you originally proposed using different technologies but switched to different ones, explain why you changed and if you believed it was the correct decision.

We originally planned to use AI to generate art assets for our game, but later switched to sprites from CraftPix.net. We changed because they were much easier to integrate into the project than the AI-generated art. Additionally, the sprites on CraftPix.net looked far better and more realistic than the ones the AI was giving us. Ultimately, we believe using the CraftPix.net sprites over AI-generated ones was the right call and helped make the game look polished for the Innovation Experience event.

GitHub and Unity had some issues working together. Each team member had some difficulty. One example is the mixture of meta files from Unity. Several times, we have had to either go through GitHub and add meta files to our .gitignore file so that it would not cause issues

when pulling the latest versions of main. Other issues occurred during the pull, causing several compilation errors. To combat this, team members had to go through Unity's "safe mode" or find workarounds to address the errors in Unity. Otherwise, it would not open the project as usual.

Design

Technical Description

Items

Many items come in many forms in Malda. Some of them are essential to travel through the world and between scenes. Others enable new mechanics. Some help the player progress in the world. Some items are to restore the player's health. All of these items share some common aspects, notably the interactable interface they all implement. Beyond that, there is significant variation in the implementations.

The interactable script is the core of the item system. Everything the player can interact with must be implemented with the interactable script to simplify the logic for player-specific mechanics. This allows new items to be created with their own scripts for how they work, without adding any code to the PlayerActions script. This enables rapid, simultaneous development of items.

A more unique interactable item in the world is the doors (*fig. 1*) in Malda. Doors are the primary means of transitioning between scenes, or levels, as the player sees them. The door's logic is to load a target scene. They're implemented in a way where they can be dragged and dropped into the scene editor, specify a scene that they're tied to, and they'll work to load that scene. There is some logic behind the scenes that enables transitions. To support a loading screen animation, the door actually sends the player to the transition scene, and the GameManager script redirects the player to the intended scene after a set time. There is also a variation of the door that is locked. The script for loading the next scene will remain unresponsive until the player interacts with the door with the required unlock items, which will be discussed later.



Fig. 1: Door frame, Portal, and Locked Portal.

The next category of items is pickups. Pickups come in multiple varieties. Pickups may boost player health during critical moments. These pickups are apples and mushrooms. Apples heal the player 1 hp, and mushrooms increase the player's max health by 1 in addition to healing 1 hp. Pickups may also be unlock items for use at a later time. Currently, these unlock items are simply keys. The keys have a simple animation that's done through code by spinning them. There are currently two keys in the game, one silver and one gold. Both keys are required to unlock a locked door in the level select hub. Pickups may also come in a variety of usable items. These usable items (*Fig. 2*) include a sword, a boomerang, and gauntlets. These items enable the player to complete additional actions after pickup. The GameManager tracks these items and their pickup status, so the player can't pick up the same item multiple times, and unlocks persistently after player death. Some items, like the boomerang, have the same model as a pickup in the world as they do when the player is actually using them. From a development standpoint, these items are effectively different. Usable item pickups enable the player to use more mechanics that are initially locked. At the current state of the game, these mechanics are the sword swing, the boomerang throw, and the ability to push crates and turrets.

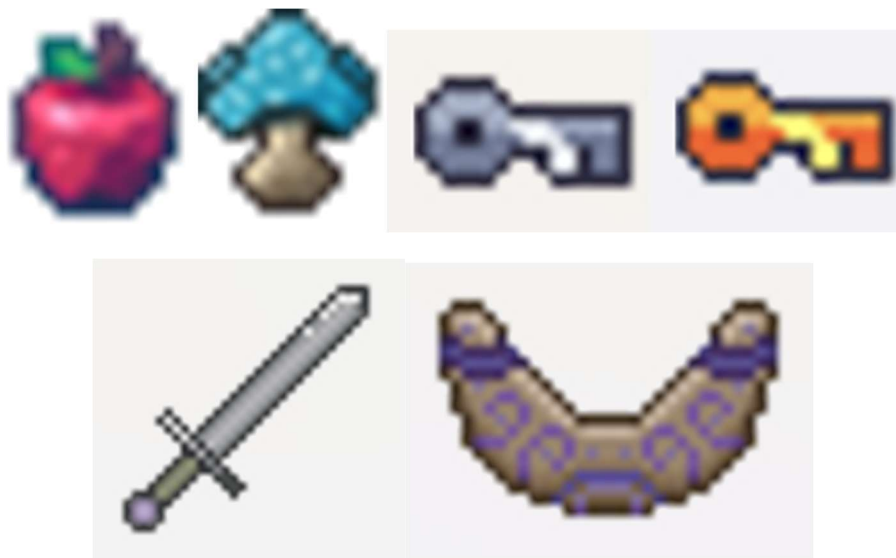


Fig. 2: Apple, Mushroom, Keys, Sword, and boomerang.

Level Design

Level design in Malda was achieved through a harmony of diverse components. The tile map tool in Unity provided a way to create physical platforms that were easy to add to and edit. The tile palette tool allows for the tile map to accept different artwork and styles. Music and sound effects are introduced at various points to showcase significance and change the feel of levels. Items, checkpoints, and enemies are added to enable the player to use their skills and experience the world in the most fun way possible. The background is also set with a parallax effect, adding a sense of depth.

In Unity, tile maps (*Fig. 3*) were the primary means of creating interactable features in the environment. The tile map enables developers to add platforms to a level in a grid. Some platforms may cover the entire grid square, and others may cover only a small section of one. These platforms have a collider for each individual in the grid. Those individual colliders are then merged to enable seamless player interaction across multiple platforms.

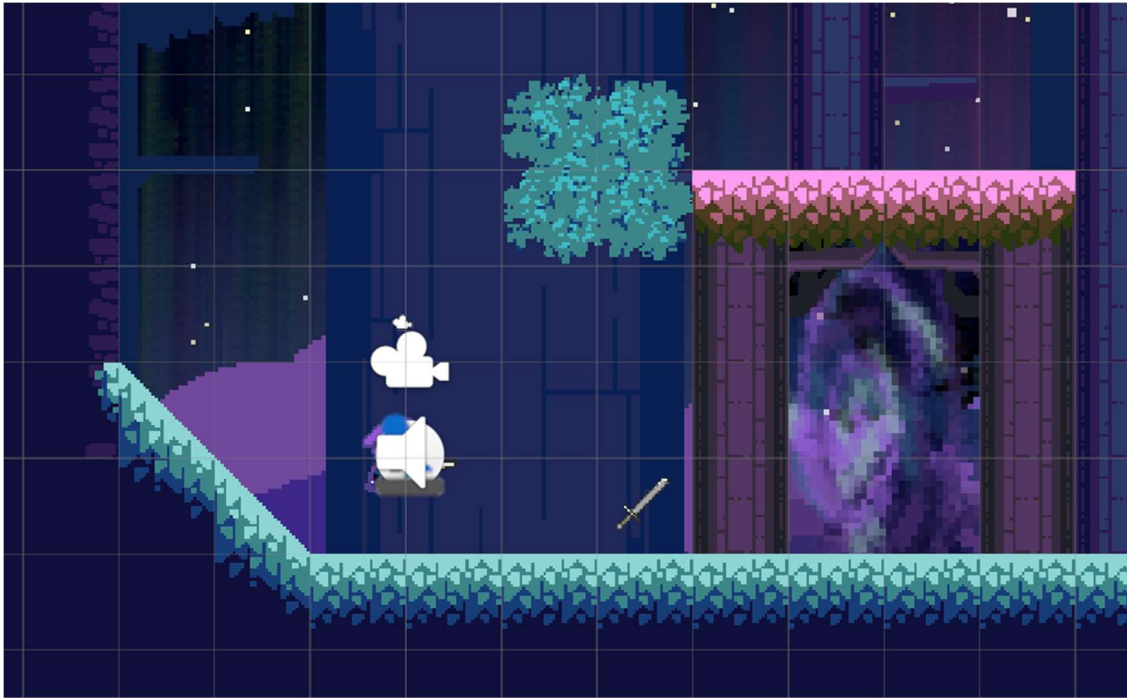


Fig. 3: Example of the tile map in grid format.

The tile palette (*Fig. 4*) tool works in harmony with Unity's tile maps. The tile palette accepts a collection of artwork for all possible tiles in a level. Once the artwork is loaded, each tile can be "painted" using a brush tool. Painting tiles of different shapes will create the platform and its associated colliders on the tile map, making the design easier.

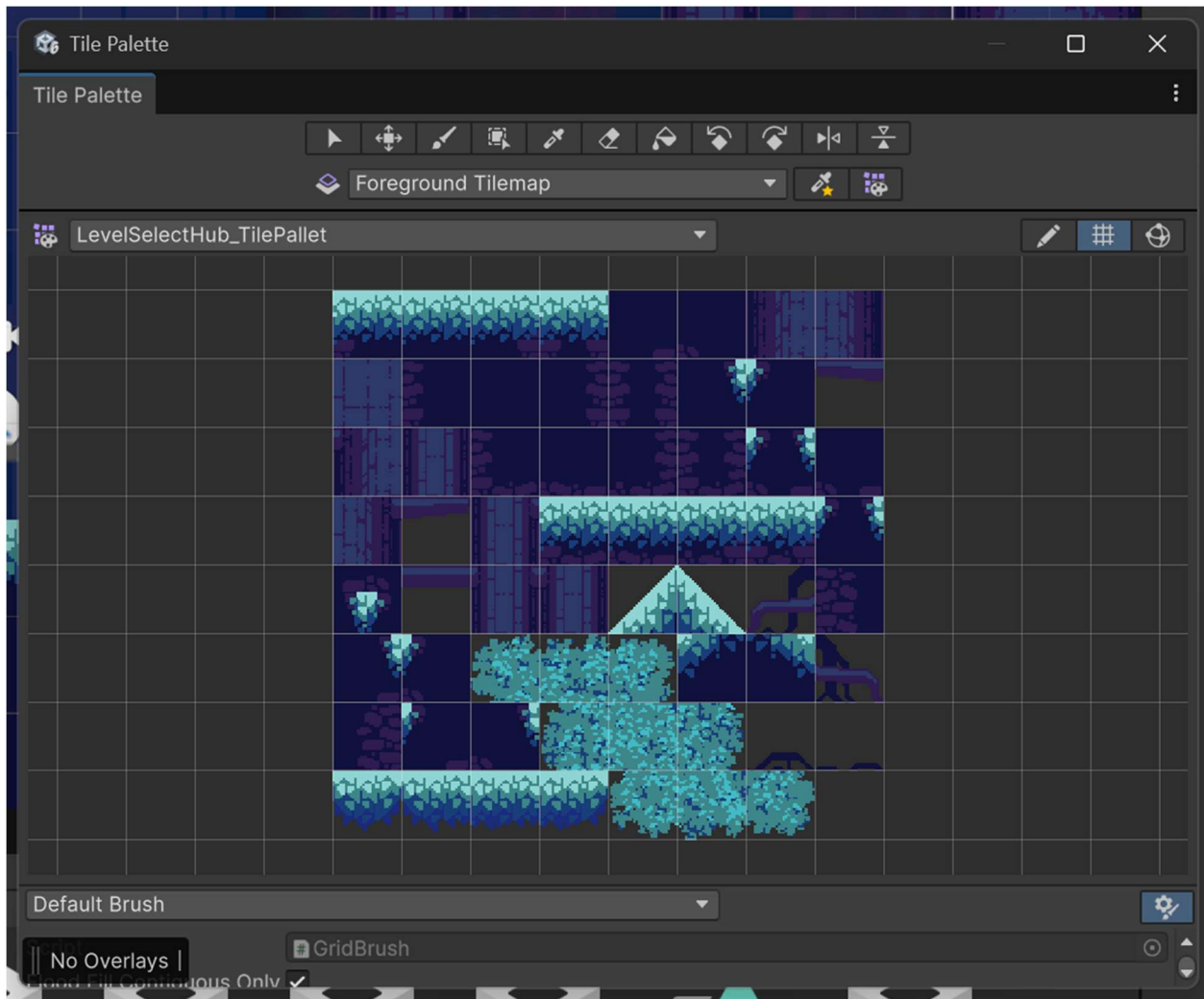


Fig. 4: Tile pallet example.

Using the tile mapping tool and the tile pallet, we added depth and detail by creating several layers labeled foreground, tilemap, and background. This gave the level the ability to add further details so that the player can really feel like they're in a 3D world, even though the game is designed in 2D. Any tile map layer used for background or foreground detailing has its colliders and ground checks turned off so the player can walk, jump, run, or perform any movement action without being boxed in by the details.

Music and sound effects are played at various locations throughout the level to make accomplishments feel significant and to provide ambiance that contributes to the narrative. Specific locations and levels trigger different music and/or sounds. Each level should have a checkpoint system that not only updates the background music for that obstacle but also sets the player's respawn point to the last known checkpoint the player had passed.

Items are added to various platforms for players to interact with, creating rewards or obstacles. (Fig. 5) There are pointers in the levels that are meant to help the player know, "Hey,

something is over there/here!” Enemies are added to situations for players to challenge and, sometimes, help solve puzzles.

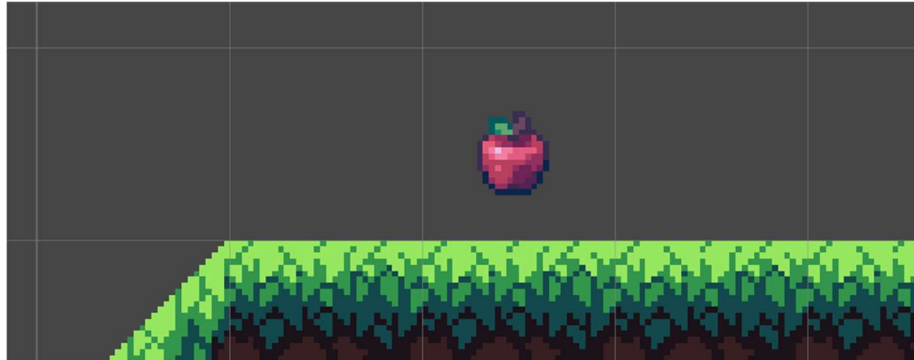


Fig. 5: An example of an area with a placed apple through the developer view.

Each level has three obstacles, ranging from easy to medium to hard. Each level in itself is also meant to be harder than the previous set of levels. The difficulty levels are also set up using a platforming method (Fig. 6), where the ground level features three equally difficult but fairly easy levels. Once the player completes those levels, they can move on to the second level of platforms, where they can reach levels 4 and 5, set at a fairly medium difficulty. The final level was not implemented in time, but level 6, at the top of the level pyramid-platform design, was the hardest. It is supposed to be designed so the player traverses vertically rather than horizontally and can defeat the big, evil bad guy in three attacks, each of which changes the vertical obstacle and makes it more difficult to traverse. The goal was to have the player smack 3 different areas of the bad guy.

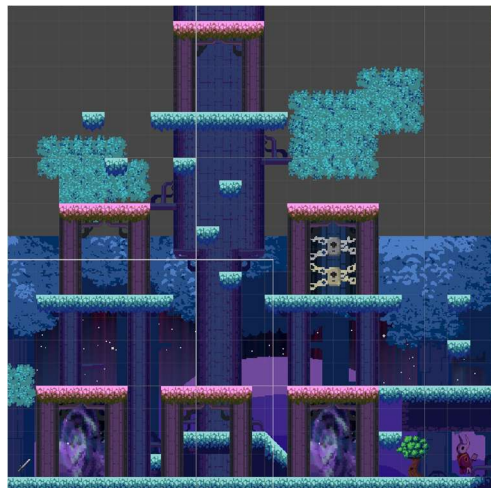


Fig. 6: Level-select hub design in developer view.

Animation

Animation and artwork are significant when telling the story of Malda and its world. Animation must feel responsive in a platforming game, so basic player interactions feel satisfying. Art and some design elements of the animation need to feel appropriate for the world. It's also critical that artwork feels “cute” and approachable to attract the target gamer demographic.

Animation was implemented on a technical level using Unity's animation controller (*Fig. 7*) and clips. Animation clips are simply different frames of an animation assigned to different moments in time throughout the clip. Each clip can have its speed adjusted and be set to loop or stop after one iteration. The animation controller takes a set of animation clips for a specific entity and handles transitions between them. Typically, transitions are timed to variables that other interactions in the world can modify. For example, when the player attacks, it triggers an animation transition to the player's jump animation. Similarly, when the player collides with the ground, a trigger is sent so that the idle animation can begin. Some transitions need to happen immediately upon an action, while others require a blend of both animations. Each transition can have any number of triggers and control the transition zone of both animation clips it's associated with.

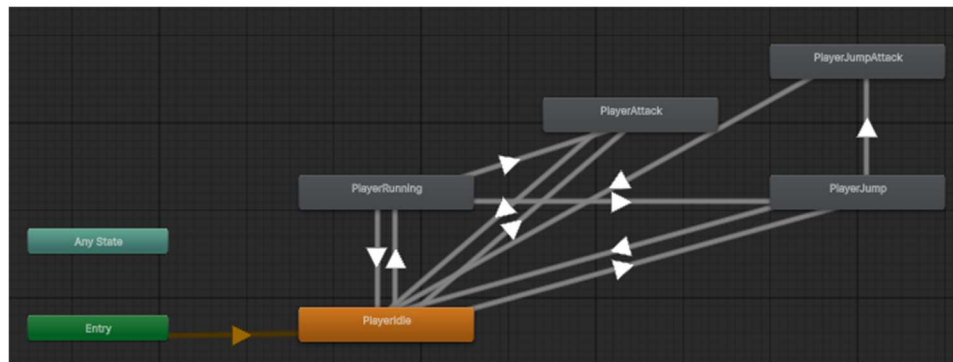


Fig. 7: Unity's Animation Controller.

Artwork in Malda is the primary means of conveying the game's style to players. The primary artwork size in the game is 32x32 pixels. This was set to ease creation on a limited timeline and to create a nostalgic feel for the game. A focus on art for the player, Malda, was required, given constant visibility throughout all aspects of the game. Otherwise, enemies needed art and animation to help explain mechanics as they interacted with players. Items needed art and some animation to stand out and draw players' attention.

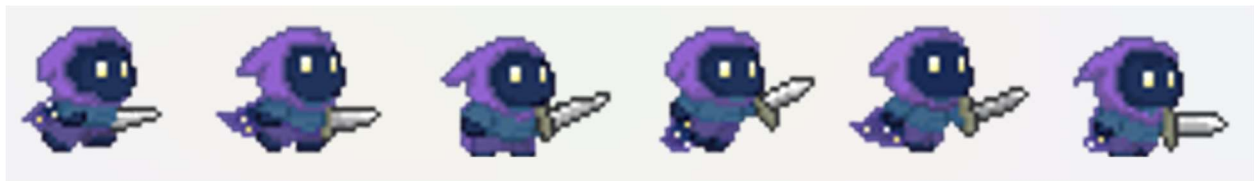


Fig. 7.5: Random sprite sheet of Malda running.

Background art was implemented to give players a feel for how they move throughout the level. This was accomplished through a parallax effect. The background was split into layers (Fig. 8), with each layer farther from the camera. This created the effect that some background layers moved faster or slower than others. The background as a whole has a very dynamic feel, with many moving parts that significantly impact player feel as they move throughout the level. This is very resource-intensive artistically, since each background layer needs to be its own sprite.

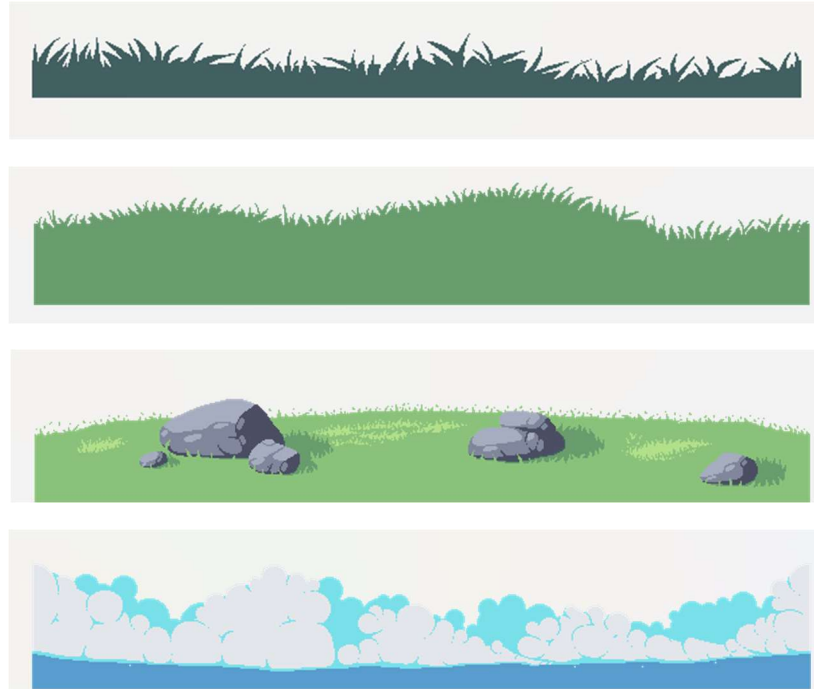


Fig. 8: Level one's background layers.

Player Logic

Controlling the playable character Malda is handled primarily through two scripts, along with various built-in Unity tools and components. Unity provides 2D Rigidbody components that handle physics and gravity, as well as the 2D Box Collider component for detecting collisions with other world objects. Keyboard and mouse input detection is handled by Unity's Input Actions system, allowing keybindings to be registered to specific actions. These components are integrated with the two-player scripts to implement all the player's actions and interactions.

Malda utilizes standard 2D platforming movement controls and feel. WASD controls the four directional movement, Space initiates a jump, and holding Shift allows the player to sprint. The player has two jumps, and all movement and attacks can be applied in the air. These keybindings are registered with keywords in Unity's Input Actions system, which broadcasts messages that the Player script can capture and handle. All movement is applied to the player with vector forces added or multiplied to the rigidbody's linear velocity attribute.



Fig. 9: Another sprite sheet of Malda, but he jumps.

Player interaction with objects in the world is a very important feature and is bound to the X key as a general interact button. Interaction is handled by overlapping 2D Box Colliders on both the player and the interactable item, triggering a message to the Player script with information about the current interactable. Interact allows the player to unlock and enter doors, pick up new weapons, and pick up consumable items that aid them throughout the game. Players must pick up the two weapons, the sword and the boomerang, before they can utilize them.

Malda has two weapons available to use throughout the game to attack and damage enemies: a Sword that can be swung with left click, and a Boomerang that can be thrown with right click. Both weapons do one damage per hit to enemies, but are useful in different scenarios. Upon using the Sword attack, a rectangular hitbox spawns in front of the player's facing direction, and when colliding with enemies, it will deal damage. Using the Boomerang attack will throw a spinning boomerang forward (*Fig. 10*), with the boomerang's hitbox following the sprite. If the boomerang hits an enemy, it will deal damage and be destroyed. Assuming no collisions destroy the boomerang beforehand, it will attempt to return to the player after a brief moment. To prevent the player from spamming attacks, a global cooldown is applied after using an attack that stops the player from using any other attack until it is refreshed.



Fig. 10: Malda threw the boomerang.

The other main script, PlayerHealth, handles Malda's health. The player starts with 3 health, and there are two consumable items the player can interact with to help manage survival. Mushroom consumables will permanently increase the player's max health, while Apple consumables will heal one point of missing health. To give players time to recover, invincibility frames are added after the player takes damage, indicated by the sprite flashing. During this time, the player cannot take any damage until they are back to a normal state. If the player's health drops to zero, the player will die and respawn at the latest respawn point.

Enemies

There are two main enemies currently in Malda for the player to interact with. The first enemy the player will run into during the first level is a simple slime (*Fig. 11*). The slime will patrol back and forth on a platform, turning around when it finds a wall or ledge. If the player runs into the slime, the player will take damage. Slimes can be damaged by either the sword attack or the boomerang. The slimes have a running and dying animation. Slimes will display their running animation as they move back and forth and play their dying animation for a few seconds after taking 3 hits from the player. While the slime is dying, the player will not take damage if they collide with the slime.



Fig. 11: The Slime.

The next enemy that the player encounters is a static turret. The turret enemy (*Fig. 12*) visually is a totem with red markings. Functionally, the totems stay in place and shoot rock projectiles at the player. The turrets have configurable range, attack speed, projectile speed, and projectile lifetime. Players can push turrets to change the position from which they shoot. The turrets can also only fire from the front. If players jump behind the turret, they will not be attacked. Turrets are positioned at various locations on levels 2 and 5, with different configurations to customize player difficulty and enable solving specific puzzles to progress.



Fig. 12: The Turret Stumps.

A script has been created, along with plans for a flying enemy that would function similarly to the slime, patrolling an area that would damage the player on contact. This enemy is currently incomplete and not implemented anywhere in the game.

Data Storage

There are three primary ways Malda stores data during the game's execution. Data can be stored locally on the player's disk, in their operating system's app data, or in a persistent memory system developed specifically for Malda.

Data that is read or written to the local disk is saved state data. For the current scope of Malda, this is quite minimal. The attributes needed to keep track of player save states are level completions and item pickups. Currently, the only position the player can save or load is the level select hub, so no location or scene instance info needs to be stored. Saving and loading are performed through a basic menu interaction from the level select hub.

The next form of data is AppData. For the most part, Unity handles this automatically. Malda currently doesn't use AppData explicitly. There are some implicit uses for AppData that Unity handles, notably with settings changes. Malda will maintain volume and resolution settings even after a player closes and relaunches the game. The Unity Engine handles those interactions. There are potentially some AppData uses for the future of Malda if user settings arise that aren't handled by the Unity Engine.

The Final form of data is persistent memory. There is a problem in Unity: loading between scenes destroys the game objects in the old scene by default. This typically means that whenever a loading screen is needed, all the information stored from the previous scene would be lost. Features like item pickups, level completions, and any world unlocks would be lost before they could be utilized. The solution to this is a GameManager script that was developed for Malda. The script, attached to a game object as all scripts need to be, changes Unity's default behavior so it doesn't delete itself when transitioning between scenes. This game object is not interactable, visible, or otherwise significant, other than simply storing data. This GameManager script stores information very similar to what would be stored on the local disk during a player's save, but also includes more niche information to assist with tasks such as scene transitions and item tracking.

How to deploy

To deploy and run the project, users must first ensure they have the required prerequisites. This includes a computer running either Windows or Linux, an internet connection, and sufficient storage space. Additionally, to install and build the project, users must install either Git or GitHub Desktop to obtain the source code, and Unity Hub with Unity Editor version 6000.3.9f1 or later to open and build the project.

The project's source code is hosted on GitHub at the following link: <https://github.com/rmohamed93/Malda>. To download the source code, users must have a GitHub account. The repository can then be cloned locally using either Git via the command line or GitHub Desktop. If using Git, navigate to the desired installation directory and execute the command "git

clone <repository_url>”, replacing <repository_url> with the URL copied from the repository page. For those using GitHub Desktop, an “Open with GitHub Desktop” option in the repository page can be used. The GitHub Desktop application will open and prompt you to select a local destination folder to clone to. After completion, the source code will be available locally using either method.

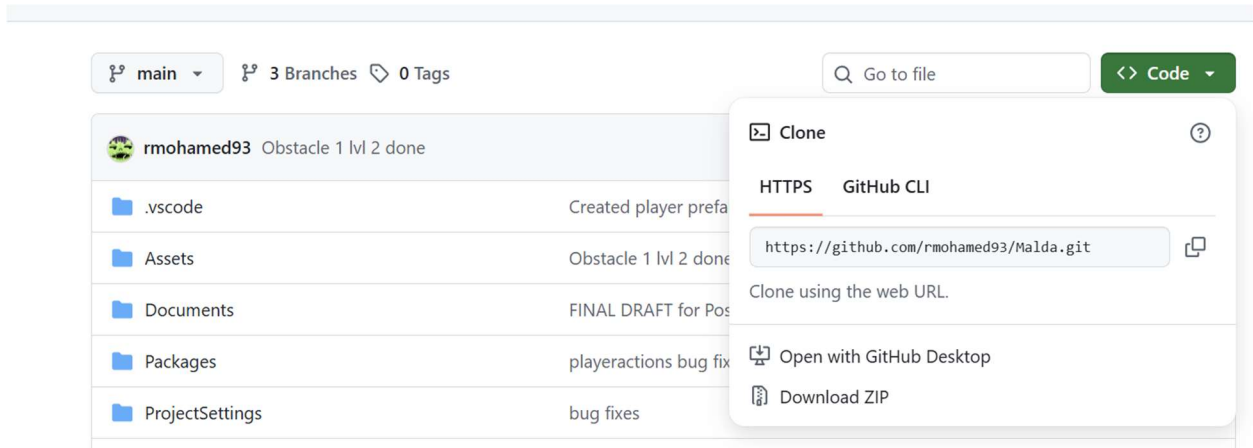


Fig. 13: Cloning the repository.

After acquiring the source code from the repository, users must then set up the project in Unity. Unity Hub can be downloaded from the following link: <https://unity.com/releases/unity-6>. A Unity account is required before installation. Within Unity Hub, navigate to the “Projects” tab, select the “Add” drop-down menu, and choose “Add project from disk.” Then, locate the folder where the repository was cloned and select it. Unity Hub may prompt you to install the required editor version or additional dependencies. Once all required components are installed, the project will open in the Unity Editor.

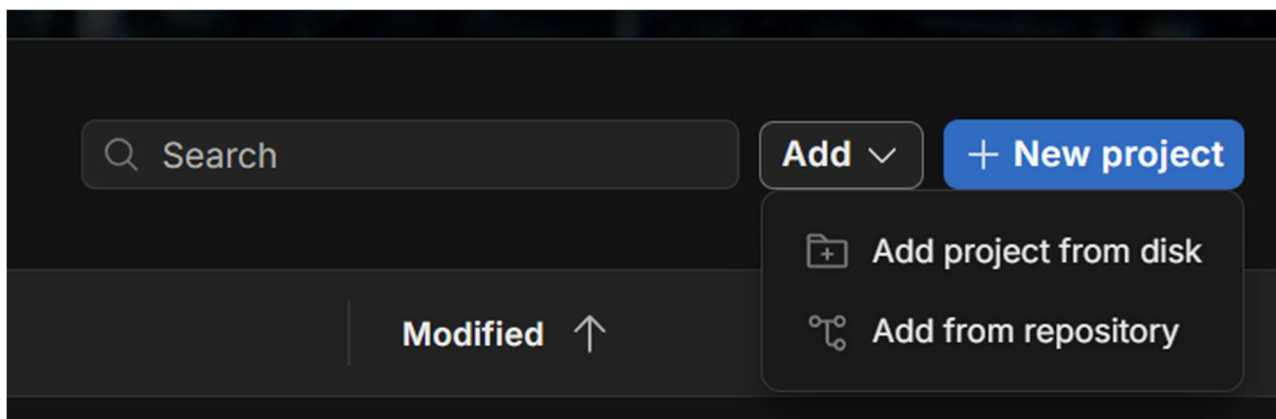


Fig. 14: Adding a new project to Unity.

As far as in-game play, our game settings open when you hit the “Esc” key, but there is a bug with our resolution option for video settings. If you change the resolution, the entire game’s saturation and brightness levels plummet, making it much harder to navigate through the game.

To build the project, users should navigate to the top menu bar in the Unity Editor and select File → Build Profiles. From there, they must select the platform they are building on. Currently, only Windows and Linux platforms are supported. Once selected, verify that all required scenes are included in the Scene List. The required scenes are: Scenes/Main, Scenes/LevelSelectHub, Scenes/Level 1, Scenes/Level 2, Scenes/Level 3, and Scenes/Level 5. If there are any missing scenes, add them by clicking “Open Scene List” and checking the missing scenes. After the build profile is set up, click “Build”, give a name to the executable, and choose or create a folder to build the project in. It is recommended to create a dedicated “Build” folder to keep everything separate. Unity will then build the project executable and any other needed files.

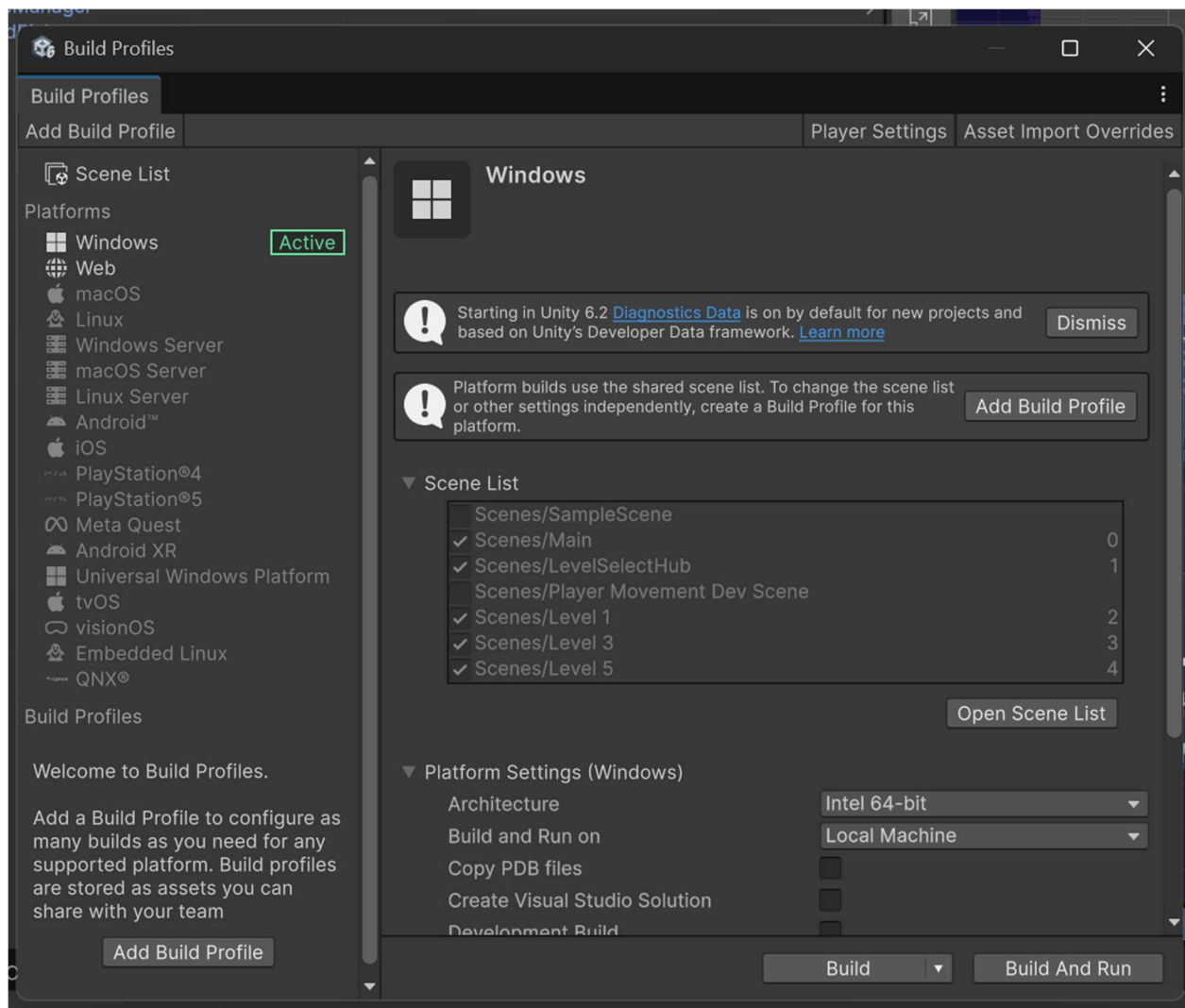


Fig. 15: Building the project to run locally.

Finally, to run the project, users can navigate to the build output folder and execute the previously-named generated file. On Windows, double-click the .exe file. On Linux systems, users may need to grant execution permissions to the file before they can double-click it or run it from the terminal. The game will then launch in full-screen mode, and users are ready to play.

Known Issues

Issues

There have been issues with respawning, where the player would respawn too quickly, causing them to merge through the ground and get stuck on the level's platforms. This issue has been difficult to fix and may still be present.

Bugs

In level one, there is a bug where the player can triple jump instead of the usual double jump. As far as in-game play, our game settings open when you hit the “Esc” key, but there is a bug with our resolution option for video settings. If you change the resolution, the entire game’s saturation and brightness levels plummet, making it much harder to navigate through the game.

Another known bug is the background sound issue. The MusicManager script doesn’t know the scene has changed, so it creates an overlapping effect with the background music from both the previous and current scenes. The only way to change it back to a single soundtrack is if the player reaches another checkpoint in any level, allowing the MusicManager to adjust.

Incomplete Features

There are a handful of features the team could not implement in time. The first was continuing our level designs and building levels two, four, and six. Levels two and four have the potential to be completed by the presentation date; they would be lacking in enemy details and other requirements as outlined in our requirements document. As mentioned earlier in the design, level six requires a vertical progression stage rather than the horizontal stages of our previous levels. It also requires special enemy scripts because, technically, our level six is the big, bad guy himself, and he appears in three different portions.

Some other features we could not design or implement in time include scene transitions, non-playable characters (NPCs), full game settings, the gauntlets, climbing boots, the Orb of Condensed Slime, and our ice beam laser. As a team, we were initially very ambitious about building the game and adding cool, interactive items for the player’s enjoyment. These were also items and game design ideas that we, as gamers ourselves, thought would be cool to have.

Scene transitions were either going to be simple fades, where if the player interacted with a door or portal that brought them to a new world or back to the level select hub, the scene would fade out to black, then the new scene would fade in. This would add a well-put-together feel and negate the jarring effect that scene changes currently have.

Our non-playable characters would be stationed throughout the levels, a.k.a. worlds, to match the given theme. The goal of the NPCs was to use storytelling, as well as guides and hints, to help the player find new ways to navigate worlds or discover hidden gems.

The settings menu and the original main menu screen had been changed to implement only a small portion of what we originally wanted. Initially, we wanted options that would allow the player to start a new game, load a previous save, open game settings, and quit the game. So far, the only items implemented are “new game” and “quit.” The other options are still there on the main menu screen, but they don’t do anything.

Gauntlets were going to be used in the level three world, where they mainly invoked a dark and dingy setting. We designed the level as a cave system and originally intended it to be a dwarven theme. Gauntlets were the objects meant to give the player a strength ability that allowed them to move heavy objects to complete puzzles and find ways around obstacles.

The slime ball object was meant to be a drop item from the slime enemies that we have placed throughout each level. This object was meant to allow the player to throw onto any ground or platform, spreading out into a small, slimy area, and to grant them a jump boost when jumping onto it.

While we have enemies with the ability to harm the player on contact, we hadn’t gotten to the implementation of an attack script for other enemies with weapons, so that when they come to the player, an animation would trigger. The enemy would attack and damage the player.

Sound effects had also not been implemented. Throughout the game, there should be basic sound effects for footsteps, jumping, sword swing attacks, and throwing items. This would have involved adding a large amount of code to the player script and creating a sound-effect manager script. The sound-effect manager script would have allowed sound effects to change based on the type of platform or ground the player came into contact with.